

Cryptography & PKI in the Real World

Companion case studies for 48730/32548 Cyber Security — Week 4 Lab: Cryptography, SSL, and X.509 Certificates

The Week 4 lab walks through five hands-on tasks: symmetric encryption with OpenSSL, asymmetric (RSA) encryption, standing up a Certificate Authority, issuing an X.509 certificate, and using that certificate to run an HTTPS web server. Every one of those steps mirrors a real mechanism that underpins the modern web — and every one of them has, at some point, failed in a real, high-profile way. This document walks through four such incidents, explains exactly how each was carried out, and ties the failure back to the specific lab task it relates to. The goal is for students to leave the lab not just knowing the OpenSSL commands, but understanding *why* each step exists and what happens when it is skipped, weakened, or abused.

What's covered

Case 1	The Adobe Breach (2013)	Task 1 — Symmetric encryption & cipher modes
Case 2	The Heartbleed Bug (2014)	Task 1 & 2 — OpenSSL implementation, key exposure
Case 3	The DigiNotar Compromise (2011)	Task 3 & 4 — Certificate Authorities & X.509 issuance
Case 4	Lenovo Superfish (2015)	Task 5 — Trusting a root CA in the browser

How to use this in your report: for each lab task, you can reference the matching case as evidence that you understand the real-world stakes behind the command you ran — e.g. when writing up Task 1, note that ECB mode's pattern-leaking weakness is exactly what let researchers recover Adobe user passwords in 2013.

The Adobe Breach

Date	Breach detected September 2013, disclosed October 2013
Who	Adobe Systems
Scale	~153 million user records, ~38 million active accounts affected
Root cause	Symmetric encryption used incorrectly — 3DES in ECB mode, single shared key, unencrypted password hints

How it was conducted

Attackers first breached Adobe's network and stole source code and, separately, a backup database containing customer records. Security journalists later found roughly 40GB of this stolen data sitting on a criminal server. The exposure came from source-code analysis that let attackers find and exploit weaknesses in Adobe's software, which gave them access to the stored account data.

The real damage, however, came from how Adobe had encrypted the passwords in that database. Instead of hashing passwords (the correct approach, using something like bcrypt or PBKDF2), Adobe had encrypted them symmetrically with 3DES in **ECB (Electronic Codebook) mode**, using the same key for every single user — the same weak setup discussed in Task 1 of this lab. Because ECB always produces the same ciphertext block for the same plaintext block, any two users who chose the same password ended up with byte-for-byte identical encrypted values. On top of that, Adobe had stored the passwords using symmetric encryption rather than hashing, and had used ECB mode with the same key for every password, which independent researchers confirmed after the breach.

Researchers didn't need to break the encryption key at all. Because the leaked database also contained each user's plaintext *password hint*, researcher Jeremi Gosney grouped users by identical ciphertext (meaning they shared a password) and then read the hints for that group. By cross-referencing thousands of hints against ciphertext groups, he was able to work out the top 100 most common Adobe passwords in about three hours — without ever recovering the encryption key.

Why it matters for this task

This is the direct, real-world consequence of the cipher-mode choice you tested in Task 1. ECB mode leaks structural information about the plaintext even while the data is technically "encrypted" — which is precisely why the lab has you compare modes like CBC, CFB, and OFB and observe that CBC (which uses an IV and chains blocks together) does not produce this pattern. Adobe's mistake shows that picking the right cipher mode isn't a technicality — it can be the entire difference between secure storage and a fully crackable database.

Sources: Cygiant (blog.cygiant.com); Graham Cluley (grahamcluley.com); CSO Online (csoonline.com).

The Heartbleed Bug

Date	Introduced 2012, publicly disclosed 7 April 2014
Who	OpenSSL project (the exact library used throughout this lab)
CVE	CVE-2014-0160
Scale	Estimated two-thirds of all HTTPS-enabled web servers worldwide

How it was conducted

Heartbleed was not a weakness in a cryptographic algorithm — it was a programming bug in OpenSSL's code, in a feature called the TLS "heartbeat" extension, which lets a client periodically check that a server connection is still alive by sending a small piece of data and asking the server to echo it back. The flawed code let an attacker retrieve private memory from an application using vulnerable OpenSSL, in chunks of up to 64 kilobytes at a time, and this could be repeated as many times as needed to harvest more memory.

The technical flaw was a missing bounds check. The vulnerable code copied a length value the client supplied without ever verifying that the length actually matched the amount of data the client had sent, so a client could ask for far more data echoed back than it had provided — and the server would pad the difference with whatever else happened to be sitting in adjacent memory. A malicious client could deliberately send a tiny heartbeat payload while claiming a much larger length, and the server would reply with the real payload plus extra, unrelated bytes pulled straight from its own memory.

Because that server memory can contain almost anything the process has recently handled, attackers could use repeated requests to silently extract passwords, private communications, and even the encryption keys used to secure HTTPS connections, all without authentication and without leaving obvious traces in server logs. Most seriously, this included the server's private key — the exact ``private.pem`` / ``server.key`` file you generated in Tasks 2 and 4 of this lab. If an attacker recovered that key from a vulnerable server's memory, they could decrypt *all* past and future traffic protected by that key, or impersonate the server entirely in a man-in-the-middle attack.

Why it matters for this task

Heartbleed is the reason the private key you generated in Task 2 (``private.pem``) and the server key in Task 4 (``server.key``) are treated as the single most sensitive artefact in the whole PKI chain. The lab's entire trust model — CA signs a cert, browser trusts the CA, server proves identity with its private key — collapses instantly if that private key ever leaks, regardless of how strong the RSA algorithm itself is. Heartbleed shows that key compromise is often not a cryptographic break at all, but a simple software bug in the library implementing the protocol.

Sources: CISA (cisa.gov); heartbleed.com (Codonomicon/NCSC-FI); CSO Online (csoonline.com); ExpressVPN (expressvpn.com).

The DigiNotar Compromise

Date	Breach began ~June/July 2011, discovered late August 2011
Who	DigiNotar (Dutch Certificate Authority, subsidiary of VASCO)
Scale	530+ fraudulent certificates issued; ~300,000 Iranian Gmail users affected
Outcome	Browsers distrusted DigiNotar; company declared bankruptcy within weeks

How it was conducted

DigiNotar was a trusted root Certificate Authority, pre-installed in every major browser's trust store — conceptually the same role your self-signed ``ca.crt`` plays in Task 3, except DigiNotar's root certificate was trusted by everyone's browser by default, not manually imported. Investigators later found that all eight of the company's certificate-issuing servers had been compromised by the intruder, and the attack had likely started more than a month before it was discovered, with evidence that log files on those same servers had been tampered with.

Once inside the CA's infrastructure, the attacker didn't need to break RSA or forge a signature cryptographically — they simply used DigiNotar's own legitimate signing process to issue certificates for domains they didn't own, exactly the same ``openssl ca ...`` step used in Task 4 of this lab, just abused from inside a compromised, trusted CA rather than a student's own local one. Hackers breached DigiNotar and issued more than 531 fraudulent certificates, including one for `*.google.com`, in a breach that started in July 2011 and was discovered on 27 August, with the company declared bankrupt only 42 days later.

With a valid-looking `*.google.com` certificate signed by a CA that browsers already trusted, the attacker could impersonate Google to any victim they could reroute traffic to. Roughly 298,140 unique IP addresses tried to reach Google websites while being redirected to look-alike sites carrying these fraudulent DigiNotar certificates, and 95 percent of those addresses originated in Iran — almost certainly so that the attacker could read the intercepted users' email. The intrusion was eventually spotted only because an ordinary user in Iran noticed a certificate warning and posted about it on a public forum.

Why it matters for this task

In Task 3 you become your own root CA and sign your own certificate with ``openssl req -new -x509``. DigiNotar shows exactly why real CAs are so heavily audited, and why their private keys (``ca.key`` in your lab) must never be exposed: whoever holds a trusted CA's private key can mint a "valid" certificate for literally any domain on the internet. It also explains why your self-signed ``ca.crt`` triggers a browser warning in Task 5 — that warning is the browser correctly refusing to extend the same blind trust to your local CA that it once extended to DigiNotar.

Sources: Threatpost (threatpost.com); Slate (slate.com); The Register (theregister.com); CFR Cyber Operations Tracker (cfr.org).

Lenovo Superfish

Date	Pre-installed Sept–Dec 2014; publicised February 2015
Who	Lenovo consumer laptops, adware vendor Superfish/Komodia
Mechanism	A self-signed root CA installed into the OS trust store, with a recoverable private key shared across all devices
Outcome	Class-action lawsuits; Microsoft Defender update to remove it; industry-wide criticism

How it was conducted

Superfish was adware pre-installed on Lenovo laptops to inject shopping ads into web pages — including pages loaded over HTTPS, which should normally be impossible without triggering a certificate warning. To get around this, Lenovo and Superfish did precisely what your Task 5 does deliberately, and taught students to recognise as a warning sign: they added a self-signed root CA to the operating system's trusted certificate store. Since Lenovo controlled the machines before sale, it had Superfish generate its own certificate and private key and add that certificate into the computer's list of trusted root Certificate Authorities, so any certificate later signed by that root would be trusted unconditionally.

From that point, the adware acted as a local man-in-the-middle proxy for every HTTPS site the user visited. Every time a Lenovo computer connected to an HTTPS website, Superfish generated a fake certificate for that specific site on the fly and signed it using its own trusted root certificate, then intercepted the connection while pretending to be the real website. Because the fake root was already trusted, the browser showed no warning at all — unlike the self-signed certificate warning you're asked to observe and then resolve in Task 5.

The scheme collapsed once security researchers examined the software. Researchers found that the private key for the root certificate was embedded directly in the software binary and protected only by a weak, hardcoded password, allowing anyone to extract it and forge valid-looking certificates for any website — and critically, the same private key was reused across every affected device. In practice this meant any attacker on the same network as a Superfish-affected laptop — for example on public Wi-Fi — could use that single extracted key to intercept and modify that user's supposedly encrypted banking, email, or login traffic without any browser warning appearing.

Why it matters for this task

Task 5 has you manually import `ca.crt` into Firefox and tick "Trust this CA to identify web sites" so the browser stops warning about `cybersec.com.au`. Superfish is the cautionary tale behind that exact checkbox: once a root certificate is trusted, the browser will silently trust *anything* signed by it, and it has no way of knowing whether that trust was granted by an informed user, an IT department, or malware quietly abusing the operating system's trust store. The lab's warning message in Task 5, before you import the certificate, is the browser doing exactly what it's supposed to — and Superfish is what happens when that warning gets bypassed at scale, by design, for millions of users who never knew it happened.

Summary: Lab Task ↔ Real-World Case

Lab Task	Concept Practised	Real-World Case	What Failed
Task 1	Symmetric encryption, cipher modes (ECB/CBC/CFB/OFB)	Adobe Breach (2013)	ECB mode leaked password patterns; ~153M records exposed
Task 2	Asymmetric (RSA) key pairs, public/private key encryption	Heartbleed (2014)	Private keys leaked from server memory via an OpenSSL bug
Task 3	Becoming a root Certificate Authority, self-signing	DigiNotar (2011)	CA servers compromised; attacker signed rogue certs at will
Task 4	Issuing an X.509 certificate via a CSR	DigiNotar (2011)	Legitimate CSR-signing process abused for domains not owned by the requester
Task 5	Trusting a CA / running HTTPS with a signed cert	Lenovo Superfish (2015)	A locally-installed root CA silently intercepted all HTTPS traffic

Key takeaway

None of these four incidents involved anyone breaking the mathematics of RSA or AES. In every case, the cryptography itself worked exactly as designed — the failure was in *how* it was implemented, configured, or trusted: the wrong cipher mode, a missing bounds check in a library, a compromised signing server, or a certificate trusted without justification. This is precisely why this lab has you perform every step by hand: understanding *why* each command, flag, and warning message exists is what prevents these exact mistakes in real deployments.